kater@finops:~$ cat talk.title

雲端把花錢的權力交給了工程師，然後呢？

雲端最危險的不是貴，而是「看不見」

kater@finops:~$ whoami

林宗穎 Kater｜玉山金控 副主任工程師

HWDC 2026 · 10/14 15:15–15:45 · F 會議室

kater@finops:~$ tmux list-windows

Agenda：今天的五個視窗

0	**緣起** WHY	**長官到底要什麼？** 上雲背景・FinOps 框架・長官的三個問題
1	**預算規劃** PLAN	**錢會不會超過預算？** 事前・事中・事後
2	**監控** MONITOR	**錢花得合不合理？** 原廠報表・即時・每日・每月
3	**FinOps 文化** CULTURE	**省錢怎麼變成習慣？** 誰花錢誰負責・Postmortem → KM・節費案例・CUD
4	**TokenOps** NEXT	**AI 的錢怎麼管？** TokenOps・自建 GPU・Next

kater@finops:~$ git log --oneline cloud/

2024 年，玉山開始上雲

timeline

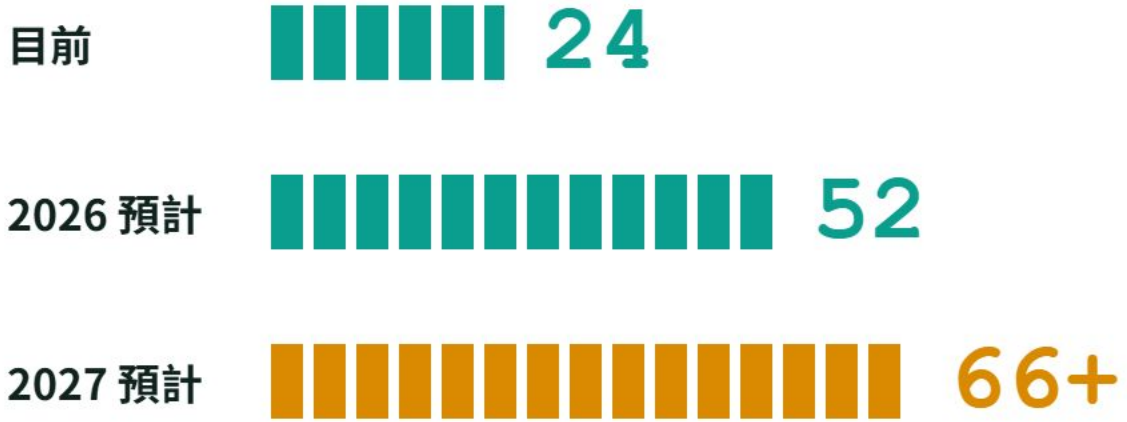
2023/08 金管會鬆綁金融上雲規範
法規開門

2023/11 董事會核可 AI 運算平台上雲
內部拍板

2024 智金處全面上雲：服務、資料、開發陸續上雲
開始動手

2026 專案數預計翻倍：24 → 52
規模化，帳單開始變成管理議題

雲端專案數量



growth
24 → 66+，約 2.75 倍

scope
每個專案 × 多個環境

專案開始萌芽，要管的帳也跟著倍增。

kater@finops:~$ gcloud projects create my-new-idea

工程師可以自己開專案、開資源

[0] 以前・地端

簽呈 → 採購 → 上架

買多少、花多少，
簽呈那一關就先審過

lead time

數週～數月

[1] 現在・雲端

```
$ gcloud projects create
$ terraform apply
Apply complete!
```

不用簽呈，
用多少、付多少

lead time

幾秒

[2] 長官在意的

這些資源的花費，
合理嗎？

之後的花費，
會不會超過
編好的預算？

帳單月底才到，
看到時錢已經花了

diff 誰都能開 · 隨時在跑 · 先用後付 ⇒ **花費從「事前審」變成「事後才知道」**

簽呈不見了，問題還在。

網路上有 FinOps 框架：對照一下，我們走到哪了？

6 原則 Principles

1 團隊要協作
2 以商業價值做技術決策
3 每個人為自己的用量負責
4 數據要即時、可取得、準確
5 由中央團隊推動
6 善用雲端的變動成本

＋FOCUS　統一各家帳單欄位的開放規格；我們以 GCP 為主，原生帳單先用好

3 階段 Phases

Inform
看得見
✓ 已建立

→

Optimize
省下來
✓ 持續做

→

Operate
持續運作
▶ 我們在這

Understand Usage & Cost

了解用量與成本　4

- ● 資料匯入 帳單匯出 BQ
- ● 成本歸屬 預算編號
- ● 報表與分析 日報、Looker
- ● 異常管理 即時告警

Quantify Business Value

量化商業價值　5

- ● 規劃與估算 架構審查
- ● 預測 月底預估
- ● 預算 三層預算
- ● 單位經濟 部分專案算到每次呼叫
- ○ 標竿比較

Optimize Usage & Cost

優化用量與成本　5

- ● 雲端架構設計 開發前、上線前審查
- ● 工作負載優化 閒置、降頻
- ● 費率優化 CUD、Spot
- ● 授權與 SaaS TokenOps
- ● 雲端永續 GCP 碳排與耗能、ESG

Manage the FinOps Practice

管理 FinOps 實務　8

- ● 實務運作 月會
- ● 政策與治理 權責分工
- ● 工具與服務 BQCA
- ● 工作負載導入 新專案估價
- ● 帳單分攤回收 分攤到各單位
- ● 教育與賦能 標準流程、Agent Skill
- ● 成熟度評估 對照框架自評
- ○ 跨框架整合

4 大領域 • 22 項能力　● 已在做 17　● 進行中 3　○ 還沒做 2

框架當地圖，不當清單：先回到長官真正要的。

kater@finops:~$ echo $BOSS_REQUEST



「花費要控管，要有報表。」

那長官到底要的是什麼？

PLAN	**預算規劃**	**事前・事中・事後** 回答：錢會不會超過預算？
MONITOR	**監控**	**即時・每日・每月** 回答：錢花得合不合理？
CULTURE	**FinOps 文化**	**誰花錢，誰負責** 回答：省錢怎麼變成習慣？

kater@finops:~$ cd ~/plan

PLAN — 1/4

預算規劃

錢會不會超過預算？

事前・事中・事後

0 緣起 ✓
1 **預算規劃**
2 監控
3 FinOps 文化
4 TokenOps

kater@finops:~/plan$ plan --before

事前：預算往下折，開發前先估價



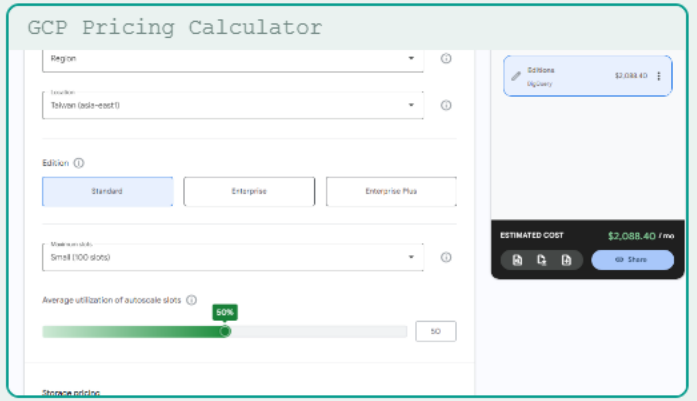
- L1 處級上限不可逾越；緩衝儲備金是最後手段
- L2 每個業管一個名目，預算管理員只盯這一層
- L3 專案預算必須編在所屬名目的額度內

開發前：架構審查

預估流量
使用者數、呼叫量、資料量

使用元件
GKE、BigQuery、AlloyDB…

↓



↓

預估月費 US$2,088 / mo

↓

← 編入所屬業管名目（左圖新專案）

事中：盯預算編號，不盯每個專案

例：某個預算編號，額度 US$100,000（示意數字）

# 專案	專案預算	目前實際花費	消耗率
A 專案	US$50,000	US$35,000	70%
B 專案	US$30,000	US$25,000	83%
合計	US$80,000	US$60,000	75%

分配率

$$\frac{\text{已編列的專案預算加總}}{\text{預算編號額度}} \times 100\%$$

80,000 / 100,000 = 80%

還剩 US$20,000 可以給新專案

消耗率

$$\frac{\text{實際花費加總}}{\text{已編列的專案預算加總}} \times 100\%$$

60,000 / 80,000 = 75%

往下看：B 專案已用 83%，最緊

超支了，處理順序

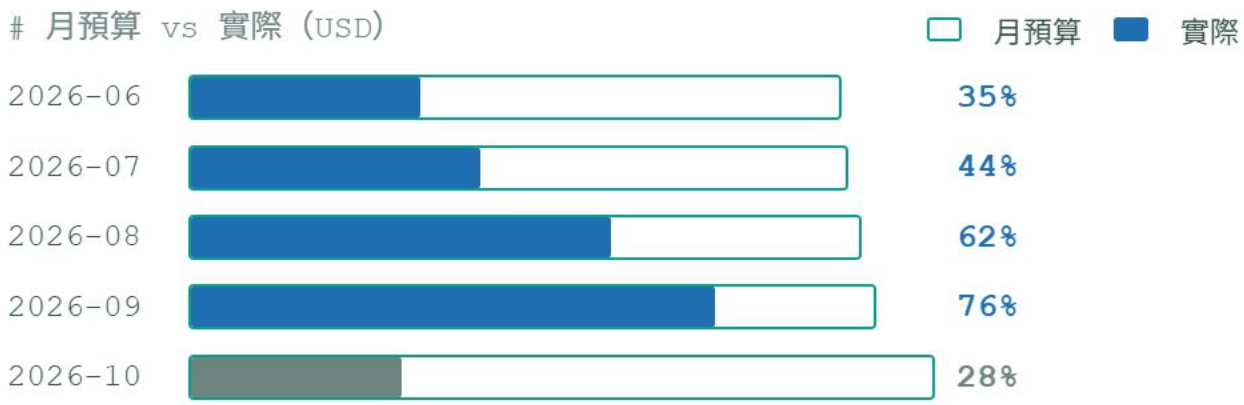
1. 止血與釐清：成長還是浪費
2. 預算編號內調度額度
3. 最後手段：動用緩衝儲備金
4. 帶到月會，重大案件做 Postmortem

預算管理員只盯預算編號，專案細節交給各自的 PM 和 TL。

kater@finops:~/plan$ diff budget/ actual/

事後：今年的實際，變成明年的預算

各專案 實際用量 | 月會 檢視落差 | 專案提報 隔年需求 | **加總成 編號額度**



月預算逐月上調（10.6 萬 → 12.2 萬），實際也追上來：35% → 76%

資料：FinOps 月報；10 月只到 10/12

由下而上

- 各專案依架構審查估價，提報隔年需求
- 同一編號加總＝隔年額度，不是由上而下攤派
- 避免「先砍再說」造成虛編

用量歷史

- 累積 6–12 個月實際用量，才決定 CUD 買多少
- 趨勢也拿來修正隔年預算



kater@finops:~$ cd ~/monitor

MONITOR — 2/4

監控

錢花得合不合理？

原廠報表・即時・每日・每月

0 緣起 ✓
1 預算規劃 ✓
2 監控
3 FinOps 文化
4 TokenOps

kater@finops:~/monitor$ open console.cloud.google.com/billing/reports

原廠帳單報表：看得到，但要知道看什麼

gcp — billing reports # Gemini 摘要＋月底預估



gcp — billing reports # 每日花費＋依服務分組



原廠都有

依服務、專案、SKU 分組

每日花費、月底預估

Gemini 自動寫摘要

但真的要用時

拉趨勢

自己切區間、換分組

看某個專案

先知道專案 ID，一個個篩

找誰超標

報表沒有預算欄，要另外對

資料都在，缺的是每天有人幫你看。

kater@finops:~/monitor/realtime$ budget-alert --list-thresholds

即時：專案預算告警，三段門檻

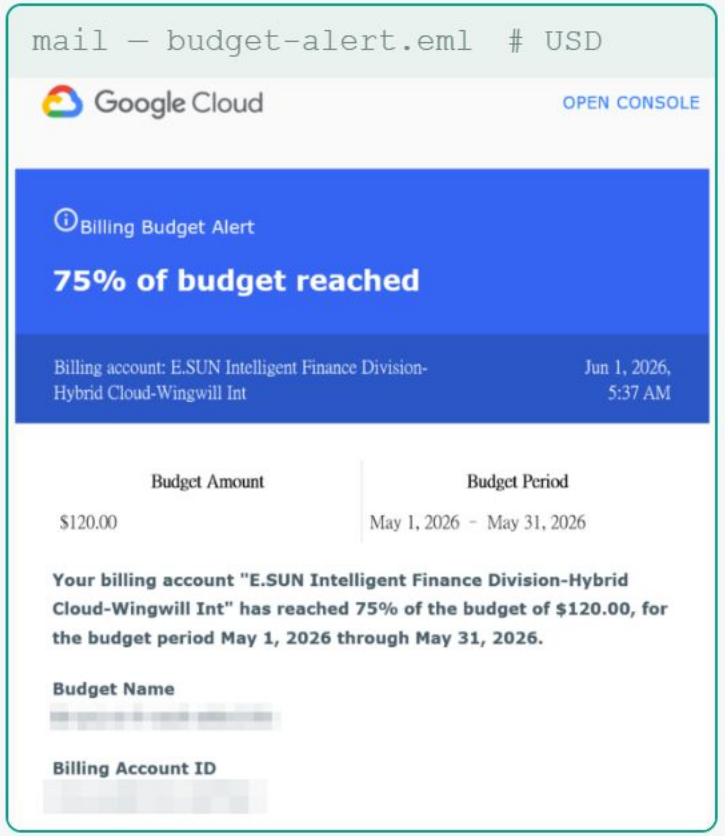
[50%] INFO 早期：負責人自查用量曲線

[75%] WARN 注意：提出後續月份用量預估

[95%] CRIT 緊急：預算管理員介入盤點

> exit 1　最嚴重：停用 Billing

雲端平台內建功能，零開發成本。回去就能先開。



kater@finops:~/monitor/daily$ tail -f overspend.log

每日①：費用超標通知，當天看見

mail — [FinOps] 雲端費用超標專案通知 # 金額單位：USD

以下為 **2026-06-05** 雲端費用超標且需要關注的專案清單（Subtotal Ratio > day/month + 5% 閾值），共 **7** 個專案： 其中 3 個大幅度飆升、 4 個本月首次超標

分類	環境	專案	TL	PM	問題	月預算	已花費	花費比例
▲ 大幅度飆升	ai				大幅度飆升，漲幅 +27.3%（45.14 → 57.47）	100	57.47	57.5%
NEW 新超標	ai				本月首次超標，花費比例 27.6%	300	82.7	27.6%
▲ 大幅度飆升	ai				大幅度飆升，漲幅 +31.8%（116.38 → 153.34）	600	153.34	25.6%
NEW 新超標	uat				本月首次超標，花費比例 25.3%	250	63.31	25.3%
▲ 大幅度飆升	uat				大幅度飆升，漲幅 +40.6%（41.29 → 58.07）	250	58.07	23.2%
NEW 新超標	ai				本月首次超標，花費比例 20.5%	100	20.53	20.5%
NEW 新超標	prod				本月首次超標，花費比例 20.0%	500	100.03	20.0%

WARN **新超標** 支出率 > 當月時間進度 + 5%

CRIT **大幅度飆升** 漲幅 ≥ 15%，標為快訊

kater@finops:~/monitor/daily$ open "FINOPS 議題追蹤.xlsx"

每日②：每一筆超標，都記進議題追蹤表

excel — FINOPS 議題追蹤.xlsx # 每月一張分頁；人名、專案名已遮蔽；金額 USD

	A	B	C	D	E	F	G	H	I	J	K	L
1	日期	專案	環境	TL	PM	問題	月預算	已花費	花費比例	回應時間	回應人員	回應
2	2026-06-16	[redacted]	ai	[redacted]	[redacted]	大幅度飆升，漲幅 +49			65.8%	2026-06-16	[redacted]	
3	2026-06-16	[redacted]	ai	[redacted]	[redacted]	大幅度飆升，漲幅 +25			105.0%			
4	2026-06-16	[redacted]	mgmt	[redacted]	[redacted]	大幅度飆升，漲幅 +15			61.7%	2026-06-17	[redacted]	
5	2026-06-17	[redacted]	ai	[redacted]	[redacted]	大幅度飆升，漲幅 +18			81.6%	2026-06-24	[redacted]	
6	2026-06-18	[redacted]	stg	[redacted]	[redacted]	大幅度飆升，漲幅 +19			215.9%			
7	2026-06-19	[redacted]	ai	[redacted]	[redacted]	本月首次超標，花費比			72.7%	2026-06-22	[redacted]	
8	2026-06-23	[redacted]	dev	[redacted]	[redacted]	大幅度飆升，漲幅 +91			87.9%	2026-06-24	[redacted]	
9	2026-06-28	[redacted]	ai	[redacted]	[redacted]	本月首次超標，花費比			103.3%	2026-06-29	[redacted]	
10	2026-06-29	[redacted]	ai	[redacted]	[redacted]	大幅度飆升，漲幅 +77			125.6%	2026-06-30	[redacted]	
11												

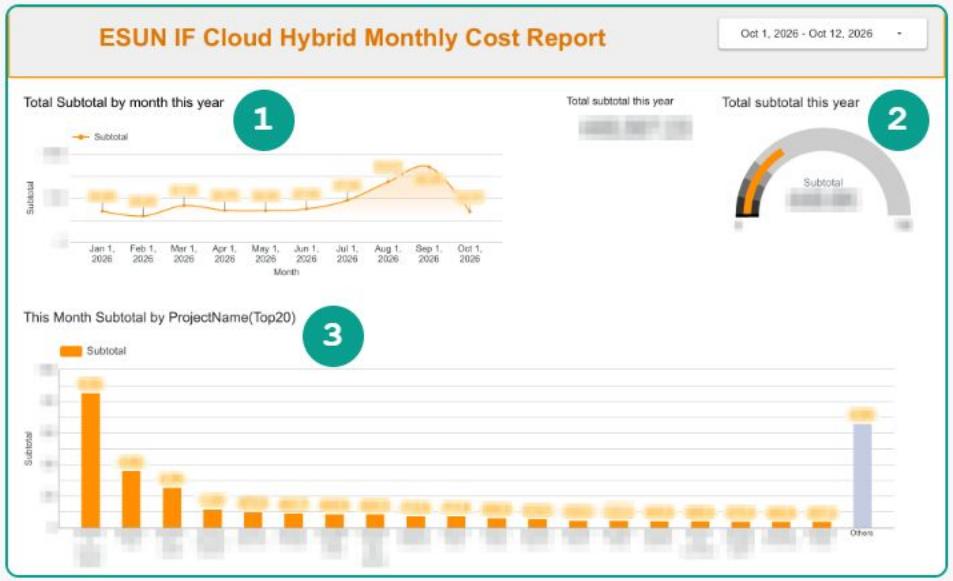
202508 | 202509 | 202510 | 202511 | 202512 | 202602 | 202603 | 202601 | 202604 | 202605 | **202606** | 202607 | 202608 | 202609 | 202610 | +

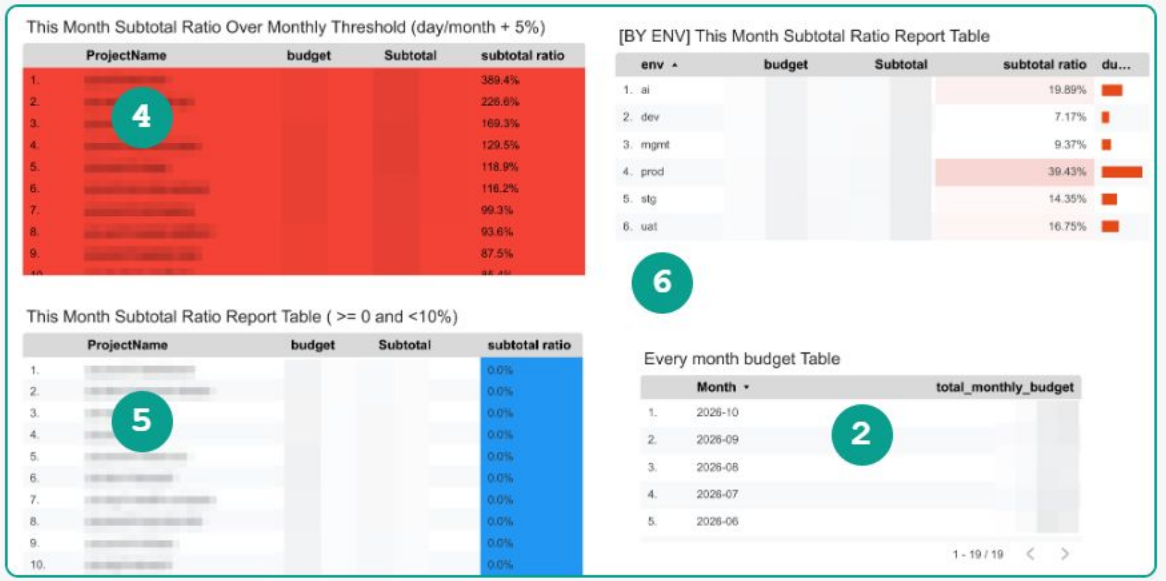
345 筆超標議題 | **15** 個月，每月一張分頁 | 回覆時間中位數 **1 天**

問題、預算、花費、誰回、怎麼回都留下來，就是月會與 KM 的素材。

kater@finops:~/monitor/daily$ crontab -l | grep report

每日③：用量日報，每天自動寄出





1 逐月趨勢

分得出是成長還是異常：9 月 US$***，是 1 月的 2.5 倍

2 年度累計＋每月預算

今年累計 US$***；月預算逐月滾動，新專案上線就調

3 本月 Top 20

前 3 名占本月近一半：要優化，先找大戶

4 超標名單

花費比例超過時間進度 +5% 自動列出，本月 20 個，直接找負責人

5 低使用率 < 10%

編了預算卻沒在用：下次編列收回，給新專案

6 各環境消耗率

正式環境本來就該花；dev／stg／uat 花多了就是浪費訊號

kater@finops:~/monitor/daily$ mail --inbox owner

每日④：閒置資源，掃到就通知

mail — GCP Idle Resource Report



GCP Idle Resource Report

專案 (Project): [redacted]

環境 (Environment): dev

閒置數量 (Count): 1 台 GCE 執行個體

詳細資訊 (Details): [redacted]

請評估是否可將這些資源關機、快照備份或刪除，以優化雲端成本。

Cloud Coach Recommendations

各位團隊成員您好！

我是 Cloud Coach 服務，這封信是要提醒您，我們偵測到您的某個 VM 已經超過 30 天沒有使用者登入了。

為了協助您進行 FinOps 成本優化，以下提供一些建議：

- 雖然 VM 已經關機，但硬碟費用仍然會持續支出 (一般 100G 硬碟或高階 SSD 每月花費分別為 $10 和 $17 USD 不等)，請先評估資源是否仍需存在。

閒置資源通知：哪台機器、停了幾天、怎麼處理

mail — 預算門檻統計



預算門檻設定統計概況

以下是截至 2025-07-15 的預算門檻設定統計數據：

項目	數值
總專案數	163
設定預算專案	163
設定預算門檻落實率(%)	100.0

未設定 Budget 的專案清單

所有專案均已設定 Budget。

預算設定落實率統計

寄出去只是開始：限期改善，回頭確認。

kater@finops:~/monitor/daily$ open looker://governance-dashboard

每日⑤：治理儀表板，資料每天更新



- 預算門檻設定落實率
- Spot VM 使用比率
- 逾 30 天未登入的 VM
- 閒置 Disk 容量

每月①：FinOps Review，主管一起看

B2-cost-structure.png # USD

B.2. GCP成本結構分析 (by service)

1. 「Cloud Monitoring」減幅35%，來自「[redacted]」在 4/10 上線 zombie process 排程自動清除機制及 metrics sampling rate 調整(10s -> 30s) 。
2. 「Apigee」重啟原因是MLaaS配合 agentic platform 專案進行些功能測試。

	11	12	01	02	03	04	4月 占比	4月 增幅	26年預估	百分比
總計										
運算 (Compute Engine, GKE Enterprise / GDC, Cloud Run, Kubernetes Engine)							28%	-9%		27%
硬體儲存 (Cloud SQL, BigQuery, Cloud Storage, Cloud Filestore, AlloyDB)							16%	6%		14%
網路傳輸(networking)							16%	-2%		13%
SaaS 非AI(API) (Security Command Center, Cloud Composer, SendGrid Email API)							9%	-5%		8%
SaaS AI(Vertax AI)							8%	-20%		8%
Notebooks							4%	-2%		1%
Cloud Monitoring							4%	-35%		2%
Cloud Logging							3%	-7%		3%
Certificate Authority Service							3%	-16%		2%
Support							4%	-6%		3%
Apigee							1%	100%		1%
其他							3%	40%		7%

成本結構：依服務逐月比較

C-overspend-stats.png # USD

C.超支統計資訊

以下為 4/1 至 4/30 統計數據：

項目	數量	占比
總project數	311	100%
「危險」(>120%)	6	2%
「警告」(100%~120%)	5	2%
「一般」(75%~100%)	13	4%
「一般」(50%~75%)	47	15%
「少量」(1%~50%)	103	33%
「極微」(0%~1%)	68	22%
「0花費」	69	22%

專案(project)	花費比例	花費/預算
[redacted]	157%	
[redacted]	134%	
[redacted]	124%	

1. [redacted]：
(1) 專案陸續加入[redacted]，主要費用來源在[redacted]及[redacted]這兩個專案的活動量；
(2) 另外發現專案有些 log 是對 troubleshooting 沒幫助，後續將研究如何推動專案避免生成這類 log 以及[redacted]分級分類的收錄機制

2. [redacted]（已說明）

超支統計：分級，逐案說明原因

主管在場：超支當場說明原因，當場決定怎麼處理。

kater@finops:~/monitor/monthly$ cat growth-or-waste.sh

每月②：是業務成長，還是水龍頭沒關？

[1]	**對照業務量**	使用者、交易量沒跟著漲，就不是成長
[2]	**多角度下鑽**	依元件／專案／環境拆開，找出哪一塊在漲
[3]	**回看紀錄**	告警紀錄，以及上月的改善措施有沒有生效
[4]	**看單位成本**	每專案、每環境的成本，而不只看總額

if	業務成長	**調整預算，評估用 CUD 鎖定折扣**
else	資源浪費	**立即修正或關閉，做 Postmortem**

kater@finops:~/monitor/monthly$ bq ask "這個專案近三個月花了多少？"

每月③：BQCA，用問的查帳

bigquery — conversational analytics # USD；專案名已遮蔽



中文提問 → 自動生成 SQL → 表格＋圖表＋文字說明

我們拿它來做的事

01 會上即問即查
報表沒涵蓋的問題，主管、PM 不用會 SQL

02 問預算狀況
接上帳單匯出表＋預算告警表：「今年預算狀況如何？」

03 找省錢機會
接上 GCP 原生建議資料，盤點能省最多的專案

04 交叉驗證
日報、儀表板的數字，換一條路再算一次

kater@finops:~/monitor/monthly$ bq ask "哪些專案能省最多？省在哪？"

每月④：用 BQCA 盤點原廠建議，找出能省的錢

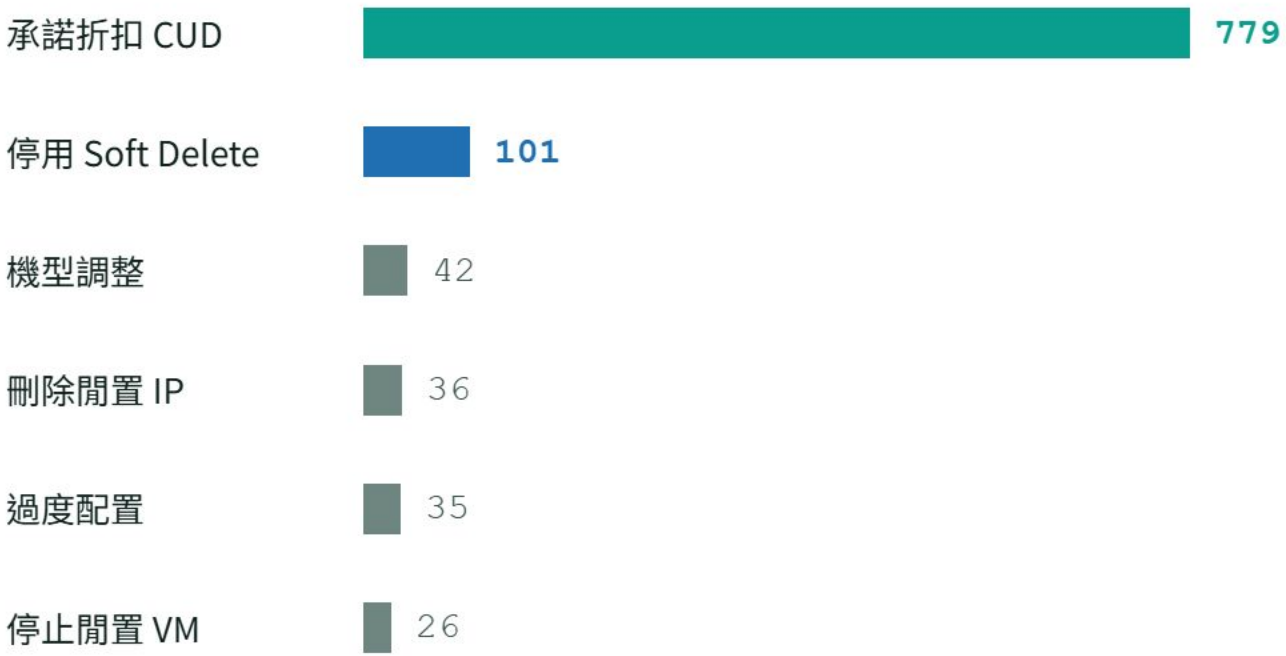
怎麼做

1 讀懂欄位
從 Console 與官方文件，弄清每個建議怎麼算

2 只看成本類
依影響類別與優先度分群，聚焦能省錢的

3 用 BQCA 統計，再跟團隊確認
避免把刻意的配置誤判成浪費

建議類型的預估節省（每月，約值，USD）



- CUD 撐起大部分節省；能省最多的前幾名，多是資料工程的共用型專案
- 停用 Soft Delete 單一 bucket 儲存成本降 71%，但刪了無法復原，先確認資料保護需求



kater@finops:~$ cd ~/culture

CULTURE — 3/4

FinOps 文化

省錢怎麼變成習慣？

誰花錢誰負責・Postmortem → KM・節費案例・CUD

0 緣起 ✓
1 預算規劃 ✓
2 監控 ✓
3 FinOps 文化
4 TokenOps



FinOps 文化：誰花錢，誰負責

雲端預算管理員	盯組織層級的分配率與消耗率，評估是否購買 CUD
中心主管	管理緩衝儲備金，確認預算編號足以支應新專案
專案負責人 PM／TL	盯自己專案的預算是否超標，回覆告警
工程師	開資源前先想成本，關掉不用的閒置資源

不究責（Blameless）：錯誤是學習的機會，不是追究的理由。



把 Postmortem 沉澱成 KM：異常 → 預防

監控告警 | 異常分析 | 改善行動 | **KM 沉澱** | **事前預防**

- 停在分享會，聽完就忘
- 沒有固定格式，深度不一
- 改善行動沒人追蹤

+ 統一模板：發生／發現／解決／預防
+ 集中存放，依服務與根因搜尋
+ 設追蹤欄，定期 Review

再往前一步：KM 變成架構審查 Checklist；最佳設定打包成 IaC 模組

kater@finops:~/optimize$ finops cases --list



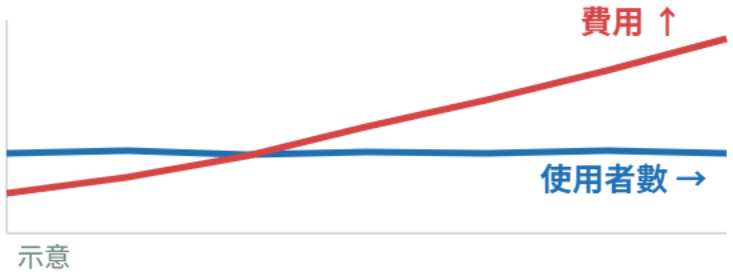
累積經驗之後：三個真實節費案例

CASE	發現方式	根因	系統改良	效益（新台幣）
智能徵信	每日超標通知	壓測資料誤輸入、異常處理設計不良	重新設計異常處理流程	NT$*** 一次性
爬蟲平台	每日超標通知	查詢語法與資料結構設計不良	重新設計資料結構、優化查詢語法	NT$***／年 省 98%
雲端虛擬桌面	月度 FinOps Review	資源閒置、未使用折扣機制	監控採樣調整，購入 CUD	NT$***／年 省 43%

三件都是機制主動發現的，不是等帳單出來才知道。

kater@finops:~/optimize/vdi$ git diff before after

案例：雲端虛擬桌面｜使用者沒變多，費用卻一直漲



示意

\# 怎麼發現的：月度 FinOps Review

告警沒響，因為費用是**慢慢漲**的。月會上把費用和使用者人數畫在一起，才看到兩條線**脫鉤**。

→ 不是業務成長，是浪費。依元件拆開，找到兩個來源 ↓

@@ ① 監控冗餘 @@

- 每台桌面的監控採樣太密、指標收太多

\+ 不影響監控成效下，調降採樣頻率

\# 監控費用跟著收集量計價：收越多，付越多

saved / year **NT$***

@@ ② 資源閒置 @@

- 正式環境整年在跑，卻用隨需價
- 測試環境沒人用時也開著

\+ 正式環境買 CUD；測試環境關閉閒置資源

\# 穩定在跑的負載才適合 CUD，後面 P30 會講

saved / year **NT$***

TOTAL　NT$*／年，省 43%**　　\# 先在測試環境壓測比對，上線後雙軌觀察；稽核紀錄不動

kater@finops:~/optimize$ git log --grep="caught by alert"

兩件被超標通知攔下的事

@@ 爬蟲平台 @@

−98% NT$*** 每年

- BigQuery 查詢語法與資料結構低效

+ 資料表加上 partition／cluster，重構 SQL

@@ 智能徵信 @@

NT$*** 當年度一次性

- 壓測資料誤輸入，Timeout 處理設計不良

+ 重新設計異常處理流程，建置 Checkpoint

功能都正常、結果都正確，所以沒有人會主動去看。

kater@finops:~/optimize$ gcloud compute commitments list

有了用量歷史，才能談 CUD

[1] 只承諾長期穩定的基載用量

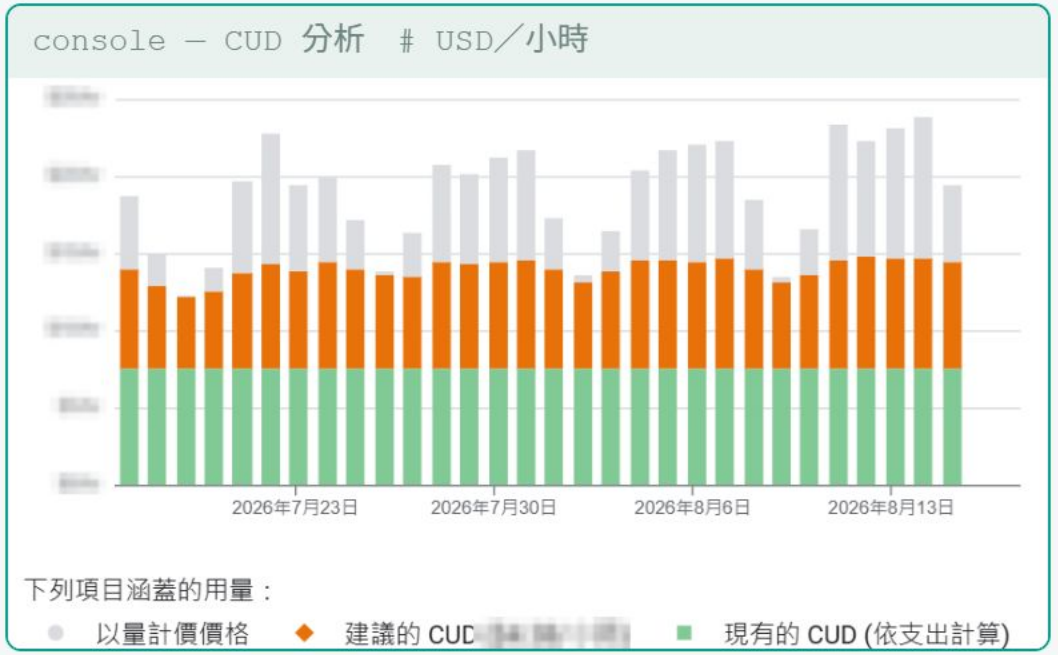
[2] 先買建議值的 80%，留緩衝

[3] 優先選彈性型，可跨機型與區域

[4] 集中持有，跨專案共用折扣

[5] 持續看使用率與覆蓋率

前提：先有 6–12 個月的用量歷史



100%

承諾使用率



kater@finops:~$ cd ~/next

NEXT — 4/4

TokenOps

AI 的錢怎麼管？

TokenOps・自建 GPU・Next

0 緣起 ✓

1 預算規劃 ✓

2 監控 ✓

3 FinOps 文化 ✓

4 **TokenOps**

更進一步：從 FinOps 到 TokenOps

AI 的成本單位，
從 VM 變成 GPU 與 token。

AI SaaS 淨成本・2026 年至今

US$*

*** tokens（token＋語音等）

雲端 GPU 淨成本・2026 年至今

US$*

17,624 GPU 小時，約一半是 Spot

#9 月 AI 單月約 US$***，是 1 月的 10 倍以上：日報那個 2.5 倍，答案在這。

looker — TokenOps Monthly Monitoring Dashboard # USD

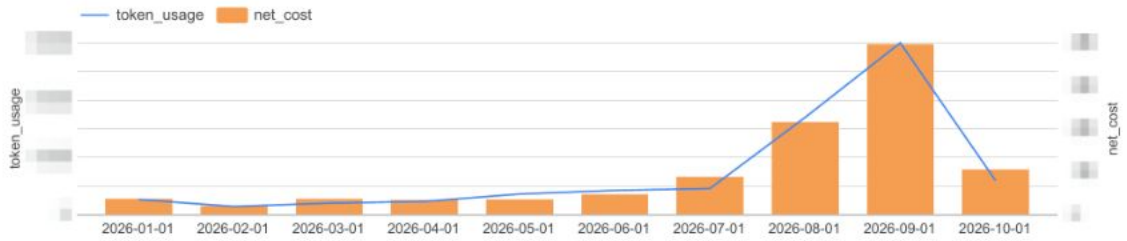
[Esun][TokenOps] Monthly Monitoring Dashboard

選取日期範圍

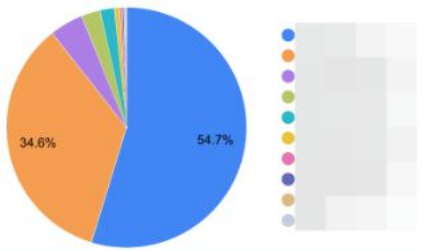
Total net cost

Total token usage (B)

Total net cost and token usage by month



Net cost by AI model



Token usage and net cost by model

	service	model_name	token_usage (b)	net_cost	token usage pct.	net cost pct.
1.	Vertex AI				52.78%	11.12%
2.	Vertex AI				13.59%	5.42%
3.	Vertex AI				5.85%	8.35%
4.	Vertex AI				5.63%	0.96%
5.	Vertex AI				5.46%	0.36%
6.	Vertex AI				5.27%	10.75%
7.	Vertex AI				5.07%	4.96%

1 - 36 / 36

kater@finops:~/next$ diff on-prem/gpu cloud/gpu

都上雲了，為什麼還要自建 GPU？

# 同樣 24 張 GPU	[0] on-prem — 自建	[1] cloud — 雲端
01 **費用**	**一次採購，按年攤提** 用得越滿，每小時成本越低	**US$***** ／月（隨需） 試算約每年 US$***；開著就計費 AI SaaS 實際：9 月單月約 US$***，隨用量長
02 **穩定與主權**	**算力握在自己手上** 服務穩定，排程與模型版本自己決定	**時不時就斷線** 限流、配額、模型下架，由供應商決定

不是二選一：TokenOps 把地端 GPU 和雲端 token 放在同一本帳上比。

kater@finops:~/next$ cat ROADMAP.md

Next：還能做什麼？AI Agent 放在哪？

# 維度	下一步	AI Agent 放這裡
預算・事前	**成本試算接進部署流程** 發 PR 就看到預估月費	**PR 成本審查 Agent** 超過專案額度先標記，請 TL 確認
監控・每日	**告警依金額分級，追回覆率** 小額併入日報，大額即時通知、追到有人回	**超標初判 Agent** 讀帳單與追蹤表歷史，先寫好可能原因
監控・每月	**建立單位成本** 每次對話、每筆交易花多少	**月報摘要 Agent** 把月報濃縮成 3–5 條待辦
文化・KM	**Postmortem 可檢索** 依服務與根因找得到	**KM Agent** 新專案上線前，比對相似的舊事故

已上線：BQCA 即問即查、Cloud Coach 閒置掃描。原則：Agent 做初版，人做決定。

kater@finops:~$ cat ANSWER.md

長官要的，其實是三個答案

錢會不會超過預算？ → **預算規劃**
事前編、事中盯、事後回饋

錢花得合不合理？ → **監控**
即時告警、每日報表、每月檢討

省錢怎麼變成習慣？ → **FinOps 文化**
誰花錢誰負責、經驗寫成 KM

先累積，再優化：三個案例、CUD，下一站是 TokenOps。



Takeaways：帶回去的四件事

01	**先讓錢看得見，再談省錢**	看不見的成本，連「貴不貴」都無從判斷
02	**監控分三種節奏**	即時守預算門檻、每日抓異常與趨勢、每月判斷成長或浪費
03	**把帳交到花錢的人手上**	告警直送 PM／TL；不究責，把經驗寫成 KM
04	**先累積，再優化**	有用量歷史才談 CUD；同一套方法延伸到 AI token

kater@finops:~$ echo $LESSON

讓每一次異常，
都成為下一次預防的養分。

權力交給了工程師，接下來要交給他們的，是看得見的數字。

kater@finops:~$ exit # Q&A 謝謝聆聽｜林宗穎 Kater